The Energy Consumption of Legacy COBOL Systems: A Forgotten Direction and COBJoules Tool

Shivadharshan S
RISHA Lab
Indian Institute of Technology
Tirupati, India
cs22b057@iittp.ac.in

Sridhar Chimalakonda
RISHA Lab
Indian Institute of Technology
Tirupati, India
ch@iittp.ac.in

Abstract

The increasing integration of artificial intelligence (AI) into everyday tasks has driven demand for high-performance software to an unprecedented level. This has resulted in an exponential growth in energy consumption by software systems. While extensive research has been done focusing on measuring and evaluating the energy consumption of machine learning and AI systems, particularly in Python, a significant gap remains in tools and methodologies for assessing energy usage in legacy systems. Legacy software systems continue to underpin critical operations across industries such as finance, government, and healthcare, yet their energy consumption remains largely unmeasured and unoptimized. As organizations face mounting pressure to improve sustainability and reduce operational costs, the lack of dedicated tools for quantifying the energy usage of legacy codebases, particularly those written in languages such as COBOL, has become a significant barrier. In this paper, we discuss this critical challenge and specifically focus on the legacy language of COBOL. In particular, we present *COBJoules* as one of the first tools designed to measure and analyze energy consumption in COBOL environments. The tool was evaluated using benchmark tasks from Rosetta Code, demonstrating its effectiveness in quantifying energy consumption patterns for COBOL. The tool and all related artifacts can be found at https://github.com/rishalab/COBJoules.

CCS Concepts

• **Software and its engineering → Software performance**; *Software verification and validation*; *Software maintenance tools.*

Keywords

Green Software, Legacy Systems, COBOL, Energy Measurement, RAPL

ACM Reference Format:
Shivadharshan S and Sridhar Chimalakonda. 2026. The Energy Consumption of Legacy COBOL Systems: A Forgotten Direction and COBJoules Tool. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE-Companion '26), April 12–18, 2026, Rio de Janeiro, Brazil.* ACM, New York, NY, USA, 2 pages. https://doi.org/10.1145/3774748.3795677

This work is licensed under a Creative Commons Attribution 4.0 International License.
ICSE-Companion '26, Rio de Janeiro, Brazil
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2296-7/2026/04
https://doi.org/10.1145/3774748.3795677

1 Why Care About Energy Consumption of Legacy COBOL Systems?

Energy efficiency has emerged as a pivotal challenge in modern software engineering. While recent research has heavily scrutinized the energy demands of AI and Machine Learning [3, 6], the energy consumption of legacy software systems has received comparatively little attention. This gap is especially significant given that legacy systems remain integral to critical sectors such as finance, healthcare, and government. Their prolonged use, often driven by operational, regulatory, and economic considerations, results in substantial aggregate energy consumption that warrants thorough investigation. This is a critical oversight given the sheer scale of legacy infrastructure, as an estimated 800 billion lines of COBOL code remain in production globally, powering systems that handle over $3 trillion in daily financial transactions.

The lack of standardized frameworks for quantifying energy use in legacy software systems further limits the scope of green computing research. Specifically, there is extensive research and availability of energy measurement tools for modern programming languages. Tools such as `pyJoules` [1], CodeCarbon, and `RJoules` [2] have enabled developers working in contemporary environments to quantify, measure, and optimize the energy consumption of their applications. However, legacy systems remain largely excluded from these advances.

While modernization initiatives, including code converters and migration frameworks, have facilitated the partial transition of some codebases to newer platforms, a vast amount of software remains operational in its original form. This continued reliance poses a significant challenge for organizations striving to meet sustainability goals, as the lack of effective energy consumption measurement tools impedes transparency and optimization efforts. By bringing attention to this forgotten direction, we aim to broaden sustainability discussions beyond modern and AI-driven systems, shedding light on the often-hidden but significant energy footprint of aging applications.

As an initial step towards addressing this gap, we introduce **COBJoules**, a tool specifically designed to measure energy usage in COBOL programs on Linux. By leveraging hardware-level power interfaces, *COBJoules* provides the visibility needed to identify energy hotspots in aging codebases.

2 Design & Implementation of COBJoules

COBJoules leverages the Intel Running Average Power Limit (RAPL) framework [5], which is implemented directly in hardware, to measure energy consumption for both CPU (Package) and DRAM components. To achieve this, it accesses detailed energy data across

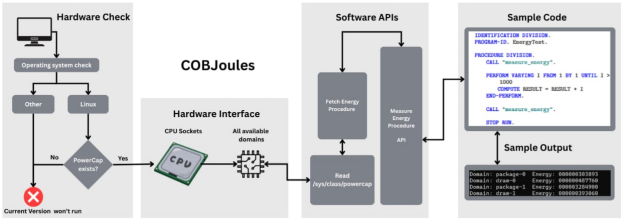

Figure 1: Execution Flow of *COBJoules*

various power domains using the `powercap` interface natively available on Linux systems [2].

2.1 Architecture & Execution Flow

The internal workflow of *COBJoules* on the host system operates through a precise sequence of hardware verifications and procedure calls, as illustrated in Figure 1.

(1) **Hardware Verification:** Upon initialization, the process begins by verifying compatibility with the host system. It locates the `/sys/class/powercap` directory, which is typically present on Linux machines equipped with modern Intel processors. Its absence indicates that *COBJoules* cannot track energy on that specific system.
(2) **Instrumentation:** To perform the measurement, the tool exposes a `measure_energy` procedure. Developers place a call to `measure_energy`, insert the target COBOL code snippet or procedure they wish to profile, and then call `measure_energy` again to conclude the tracking.
(3) **Calculation and Output:** The first procedure call captures an initial baseline reading from the model-specific registers. The second call captures the final reading. The tool then calculates total energy consumption by subtracting these readings, a well-established approach validated in prior empirical software engineering research [2]. Finally, the calculated results in Joules are printed directly to the terminal.

3 Evaluation

Given the absence of a *ground truth* for COBOL energy measurement, we adopted a methodology analogous to `RJoules` [2], validating *COBJoules* by comparing its energy trend measurements against `pyJoules`, a verified Python tool.

3.1 Experimental Setup

We utilized the **Rosetta Code** repository to select four CPU intensive tasks: *Array Concatenation*, *Bubble Sort*, *Insertion Sort*, and *N-Body Simulation*. These tasks were selected based on prior work identifying their suitability for energy benchmarking [4]. Experiments were conducted on an Ubuntu system equipped with an Intel Xeon Gold 6226R CPU (2.90 GHz) and 128 GB RAM. To mitigate system noise, each task was executed ten times with a 30-second idle period between iterations to prevent power tail states.

3.2 Results

We analyzed the mean energy consumption for both Package (PKG) and DRAM domains. The results, summarized in Table 1, reveal two key insights:

1. Trend Alignment: The energy ranking of tasks was identical for both languages (Bubble Sort < Array Concat < Insertion Sort < N-Body). We calculated Pearson's Correlation Coefficient across the tasks, resulting in values consistently exceeding **0.99** ($p < 0.001$). This near-perfect correlation confirms that *COBJoules* accurately captures the relative energy cost of computational workloads.

2. Efficiency of Legacy Code: COBOL consistently consumed significantly less energy than Python. For instance, the N-Body simulation required ≈41,596 J in Python but only ≈14,959 J in COBOL. This is attributed to Python's interpreted overhead versus COBOL's compiled efficiency.

Table 1: Comparison of Mean Energy Consumption (Joules) for Python and COBOL across different tasks.

	PKG Energy (J)		DRAM Energy (J)	
Tasks	**Python**	**COBOL**	**Python**	**COBOL**
array_concat	869.72	53.84	71.96	42.68
bubble_sort	1279.18	164.27	94.84	12.22
insertion_sort	386.46	204.92	29.38	2.00
n_body	41596.31	14959.44	162725.08	25980.20

4 Conclusion & Future Work

This work highlights the overlooked importance of energy profiling in legacy systems. *COBJoules* demonstrates that accurate energy measurement is feasible for COBOL environments using standard hardware interfaces. Our results not only validate the tool against modern baselines but also reinforce the potential energy benefits of maintaining optimized legacy code.

Future work will focus on extending support to IBM z/OS mainframes, where a significant portion of global COBOL workloads reside. We also aim to develop metrics that account for the hybrid nature of modernizing applications, where legacy backends interact with AI-driven frontends.

References

[1] Mohammed chakib Belgaid, Romain Rouvoy, and Lionel Seinturier. 2019. *Pyjoules: Python library that measures python code snippets.* https://github.com/powerapi-ng/pyJoules
[2] Rajrupa Chattaraj and Sridhar Chimalakonda. 2023. RJoules: An Energy Measurement Tool for R. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 2026–2029.
[3] Stefanos Georgiou, Maria Kechagia, Tushar Sharma, Federica Sarro, and Ying Zou. 2022. Green ai: Do deep learning frameworks have different costs?. In *Proceedings of the 44th International Conference on Software Engineering*. 1082–1094.
[4] Stefanos Georgiou, Maria Kechagia, and Diomidis Spinellis. 2017. Analyzing programming languages' energy consumption: An empirical study. In *Proceedings of the 21st Pan-Hellenic Conference on Informatics*. 1–6.
[5] Kashif Nizam Khan, Mikael Hirki, Tapio Niemi, Jukka K Nurminen, and Zhonghong Ou. 2018. RAPL in Action: Experiences in Using RAPL for Power measurements. *ACM TOMPECS* 3, 2 (2018), 1–26.
[6] Roberto Verdecchia, Luís Cruz, June Sallou, Michelle Lin, James Wickenden, and Estelle Hotellier. 2022. Data-centric green ai an exploratory empirical study. In *2022 International Conference on ICT for Sustainability (ICT4S)*. IEEE, 35–45.